Appello del 12 Settembe 2018



Politecnico di Milano
Anno accademico 2017-2018

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Cugola	☐ Mottola	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, smartwatch, ...).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. La consegna di un compito gravemente insufficiente comporta l'impossibilità a partecipare all'appello successivo.
6. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Si consideri la seguente specifica di una classe `Sequence`.

```
public class Sequence<T>    {
   //una sequenza di elementi di tipo T.

   //inizializza alla sequenza vuota
   public Sequence();

   //inserisce info all'inizio della sequenza ipotizzando che key non esista
   public void enter(String key, T info):

   //cancella l'elemento con chiave key
   public void delete(String key);

   //modifica l'info dell'elemento con chiave key ipotizzando che tale elemento esista
   public void change(String key, T info);

   //restituisce l'elemento con chiave key. Restituisce null se key non e' presente
   public /*@ pure @*/ T lookup(String key)

   //restituisce tutte le chiavi nell'ordine della sequenza
   public /*@ pure @*/ ArrayList<String> allKeys();

   //restituisce il numero di elementi nella sequenza
   public /*@ pure @*/ int size();

}
```

Domanda a)

Si scriva un invariante pubblico per la classe. Si specifichino inoltre in JML i metodi `enter` e `change`. Si argomenti che l'invariante è effettivamente verificato.

Soluzione

```
public invariant  (\forall String key; allKeys().contains(key);
                          lookup(key)!= null &&
                          (\forall key'; allKeys().contains(key'); key'!=key;))

//@requires key!= null && T!= null && !allKeys().contains(key);
//@ensures  lookup(key)==info && allKeys().get(0)==key &&
            allKeys().size()==\old(allKeys().size()+1) &&.
            (\forall key'; \old(allKeys().contains(key'));
                     allKeys().indexOf(key')==1+\old(allKeys().indexOf(key')) &&
                     lookup(key')==\old(lookup(key'));
public void enter(String key, T info):

//@requires key!= null && allKeys().contains(key);
//@ensures  lookup(key)==info &&
            allKeys().size()==\old(allKeys().size()) &&.
           (\forall key'; \old(allKeys().contains(key'));
                     allKeys().indexOf(key')==\old(allKeys().indexOf(key')) &&
                     key!=key' ==> lookup(key')==\old(lookup(key'));
public void change(String key, T info);
```

Il costruttore costruisce una sequenza vuota, che verifica banalmente l'invariante. Gli unici metodi che potrebbero inserire un duplicato sono `enter, change`, ma la loro requires lo impedisce. Le loro postcondizioni includono l'asserzione lookup(key)==info, lasciando poi immutato il resto della sequenza, quindi anche la condizione `lookup(key)!= nullchange` e' verificata.

A rigore, servirebbe conoscere anche la specifica di delete, che non e' qui precisata.

Domanda b)

Si consideri la seguente implementazione parziale della classe `Sequence<T>`.

```
class Nodo<T> {
  protected String key;
  protected T info;
  protected Nodo<T> next;
}

public class Sequence<T>  {
  protected Nodo<T> first;
  protected size;

  public Sequence() {first=null; size=0}

  public void enter(String key, T info) {
    Nodo<T> nuovo = new Nodo();
    nuovo.key = key;
    nuovo.info = info;
    nuovo.next = first;
    first=nuovo;
    size++;
  }

  public size() {return size:}
...
}
```

Scrivere l'invariante di rappresentazione della classe, ove possibile usando JML.

Soluzione

```
private invariant size==0 <==> first == null &&
    size>0 ==> (* il numero di elementi linkati a partire da first e' uguale a size *) &&
                (* non vi sono due elementi con lo stesso next *)
```

Domanda c)

Si vuole definire una classe `SequenzaOrdinata`, che ha la stessa interfaccia di `Sequence`, ma in cui gli elementi sono disposti secondo l'ordine lessicografico delle chiavi anzichè secondo l'ordine di arrivo.

E' possibile definire la classe come estensione di `Sequence` rispettando il principio di sostituzione di Liskov? e viceversa? Giustificare la risposta.

Soluzione

Non è possibile, in quanto la specifica di `enter` per la nuova classe ("inserire in ordine") è incompatibile con quella di Sequence ("inserire nella prima posizione").

Esercizio 2

Si consideri il seguente snippet di codice.

```
import java.util.Vector;
public class Driver {
    private StringContainer b = null;

    public static void main(String[] args){
        Driver d = new Driver();
        d.run();
```

```
    }
    public void run() {
        b = new StringContainer();
        b.add("One");
        b.add("Two");
        b.remove("One");
    }
}

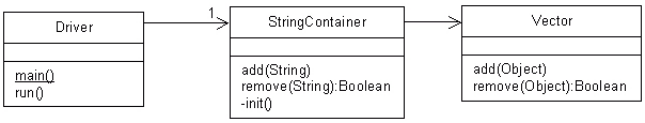
class StringContainer {
    private Vector v = null;

    public void add(String s) {
        init();
        v.add(s);
    }
    public boolean remove(String s) {
        init();
        return v.remove(s);
    }
    private void init() {
        if (v == null)
            v = new Vector();
    }
}
```

Domanda a)

Si disegni un class diagram che esprime le relazioni strutturali tra tutte le classi coinvolte nell'esecuzione del programma.

Soluzione



Domanda b)

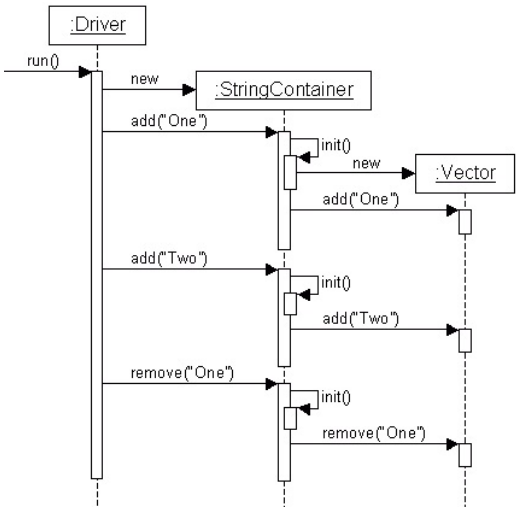
Si disegni un sequence diagram consistente con il class diagram precedente che illustri la specifica esecuzione descritta nello snippet di codice.

Soluzione

Esercizio 3

Si considerino le seguenti dichiarazioni di classi Java.

```
abstract class Oggetto {
    public abstract void parla();
}
class OggettoVolante extends Oggetto {
    public void parla() { System.out.println("Ciao, sono un oggetto volante"); }
    public void parla(Oggetto altro) {
        System.out.println("Ciao, chi sei?");
        altro.parla();
    }
}
```




```
class Uccellino extends OggettoVolante {
    public void parla() { System.out.println("Ciao, sono un uccellino"); }
}
class UFO extends OggettoVolante {
    public void parla(OggettoVolante altro) {
        System.out.println("Ciao, sono un oggetto volante non identificato, tu?");
        altro.parla();
    }
}
```

Si consideri poi il seguente frammento di codice Java che utilizza le classi sopra.

```
1.   Oggetto o1, o2;
2.   OggettoVolante o3;
3.   UFO o4;
4.   o1 = new Oggetto();
5.   o2 = new OggettoVolante();
6.   o3 = new UFO();
7.   o4 = o3;
8.   o2.parla();
9.   o2.parla(o3);
10.  o3.parla(o2);
11.  o4 = (UFO) o2;
12.  o4.parla();
13.  o4 = (UFO) o3;
14.  o4.parla();
15.  o4.parla(o4);
```

Domanda a)

Si indichino dapprima quali righe del codice sopra riportato generano un errore in fase di compilazione e quali un errore in fase di esecuzione, motivando brevemene la propria risposta.

Soluzione

- La riga 4 non compila: `Oggetto` è una classe astratta e non può essere istanziata
- La riga 7 non compila: `o3` è staticamente un `OggettoVolante`, sopraclasse di UFO.
- La riga 9 non compila: `o2` è staticamente di classe `Oggetto` e tale classe possiede il solo metodo `parla()` senza parametri.

- La riga 11 compila ma solleva un'eccezione a run-time: `o2` potenzialmente potrebbe contenere un riferimento ad un UFO e per questo il compilatore accetta il cast, ma a tempo di esecuzione si verifica che l'oggetto effettivamente riferito da `o2` sia di classe `UFO` e non essendo così viene sollevata un'eccezione.

Domanda b)

Supponendo che tutte le righe che provocano errori in fase di compilazione e/o di esecuzione siano state rimosse, illustrare l'output del programma, motivando brevemente la propria risposta.

Soluzione

Supponiamo di eliminare le righe 4,7,9,11 e anche la riga 12 che dipende dalla 11. Consideriamo le righe rimanenti ottenendo:

```
8.   Ciao, sono un oggetto volante
10.  Ciao, chi sei?
     Ciao, sono un oggetto volante
14.  Ciao, sono un oggetto volante
15.  Ciao, sono un oggetto volante non identificato, tu?
     Ciao, sono un oggetto volante
```

Esercizio 4

Si consideri il seguente codice sorgente, in un linguaggio pseudo C.

```
while (a<2) {
     if (b<1) {
        c++;
     }
     if (a>b) {
        b=a;
     }
     else if (c==3) {
        c++;
     }
     else {
        a++;
     }
}
```

Domanda a)

Si identifichi un insieme minimo di casi di test che garantisca copertura completa delle istruzioni. Tra tutti i possibili insiemi minimi di casi di test che offrono tale garanzia se ne privilegi uno che minimizza anche il numero di iterazioni del ciclo while.

Soluzione

Esaminando il codice sorgente è possibile notare che il secondo `if` ha tre possibili diramazioni, che sono quindi in mutua esclusione tra loro. Se vogliamo puntare a un unico caso di test saranno quindi necessarie *almeno* tre diverse iterazioni del ciclo, ciascuna delle quali deve essere studiata in maniera da entrare in una diversa diramazione del secondo `if`. Ciò si può ottenere ad esempio con il seguente test case: {`a=0; b=0; c=2`}.

Domanda b)

Per l'insieme di casi di test individuato al punto precedente calcolare la percentuale di copertura secondo (1) il criterio delle diramazioni e (2) il criterio delle condizioni.

Soluzione

Secondo il criterio delle diramazioni, il test case sopra ha copertura del 100%. Infatti, è progettato per coprire le tre diramazioni del secondo `if`, ma nell'ultima iterazione sarà `b=1`, per cui anche il ramo `else` del primo `if` viene coperto. Non essendovi condizioni composte, il test case ha copertura totale anche secondo il criterio delle condizioni.